Selected Topics on Deep Learning over Structured Data: **Homework 1**

# Designing a Relational Prediction Task

*Note: AI usage is encouraged, but you will be held accountable for everything you submit (code and text) as if you wrote it yourself.*

In this task you will design a relational database and prediction task, and evaluate prediction baselines on the task. The resulting database and task will be reused in Homework 2.
Any questions should be asked in the course Piazza:
https://piazza.com/class/mw0ov90gwl915r/

## Dataset

Find a real-world dataset from Kaggle, or from another source of your choosing. If you choose an alternative source, ask for approval as soon as possible. Either way, the source of the dataset should be accessible for our review. Avoid any dataset that is already in RelBench, 4DBInfer [edited:] or the Join.

## Relational Schema

Organize the dataset as a relational database with meaningful relationships between tables. Requirements:

- At least 5 tables.
- The schema graph cannot be too simple (e.g., a star).
- The schema should have multiple data types: numeric, categorical, text, and dates.
- The data should contain at least 100K tuples overall. See also the sizes of the target tables listed below.

You may choose to divide to different tables than in the original source of the data, to remove some of the columns, or to clean the data in other ways. Document your choices in the report.

## Prediction Task

- Define an entity-level classification or regression task with a temporal component, where the target is predicted from information available before a specified prediction time.
- Define and construct the target table, with at least 10K target rows.
- Design a temporal train/validation/test split. Consider how to prevent temporal leakage. The validation and test should contain at least 1K rows each.
- For classification tasks, make sure that the classes are relatively balanced (no class below 10%).
- The evaluation metrics should be AUC for classification tasks, MAE and nMAE for regression tasks (as shown in the first tutorial).

**Checkpoint (Due by 15/11/26):** submit the proposed dataset, schema and task to us for approval via this form. We expect to see different datasets for each student. We may ask you to replace your choice if it is chosen by others.

## Baseline Performance

1. Historical majority/mean. For each entity, collect the past records from the target table. For classification tasks, compute the majority label; for regression, compute the mean. If there are no historical records for the entity, use a global historical majority/mean. Use that as the prediction for the new instance of this entity.
2. XGBoost on the target table. Train an XGBoost model on the target table (which should contain only the entity and timestamp), and evaluate its performance.
3. Deep feature synthesis. Construct join-aggregate features, both manually and automatically (e.g., using Featuretools or your own code). Train and evaluate an XGBoost prediction model. The expectation is that these features improve over the historical baseline and the target-table prediction (without additional features).

## Analysis

- Verify that the task is neither trivial (very good performance) nor essentially random (very bad performance). If it is, choose a different task, because it will not be usable in Homework 2.
- Explain why the database provides an interesting setting for structured ML: why taking the relational structure into account is expected to yield better results.

## What to Submit

- **Dataset files** in the form of a relbench dataset, as detailed in RelBench documentation. Specifically, there should be a parquet file per table, a manifest.yaml file detailing the schema, a sub directory for the task, with its own manifest.yaml file and a parquet file for train, val and test. Make sure that your files can be loaded as a relbench dataset, using the supplied `load_example.py`.
- All of your **code**. Specifically, you should have a `prepare_data.py` file, which runs the data preparation pipeline and can be run as follows:
  `python prepare_data.py --raw_dir path_to_raw_files_dir --out_dir output_dir`
  Organize the rest of your code according to your convenience.
- A **report**, using this overleaf template. **Page limit:** up to 5 pages, not including references. The graders are only required to read the first 5 pages. You may also include an appendix of up to 2 pages, for completeness or extra details, but the graders are not required to read it. The main body of the report should include:
  - **The data.** What the dataset represents, how you decided to design the schema. Include a schema diagram as svg. You can generate this based on the yaml file as explained here. Also include a link to the source of the dataset.

- **Dataset statistics.** Rows per table, the time span, and how many columns there are of each type.
- **The task**. What is predicted, how you chose the time cutoffs, and any other design choices.
- **Label distribution per split.** Class balance, or the target's distribution for regression, in train, validation and test.
- **Additional features.** The synthetic features you generated, how you designed and implemented them.
- **Temporal Leakage Handling.** What steps you took to avoid temporal leakage throughout the process. This should also include columns you kept even though they might reflect the future (e.g., customer's age), if there are any, and why that's acceptable.
- **Baseline results.** A table detailing the results of the baselines on the prediction task.
- **Discussion** of results. Also include an explanation of why the relational structure is expected to yield better results for the dataset you chose.
- **AI-tool use**. Where was AI used in your submission and how you verified its output.

Good luck!